INDIVIDUAL CONTRIBUTION REPORT | Member - 3

Student name	Hasiru Yasod de silva
Student ID | UOW Number	20240969 | w2181832
Group number	Group 05
Member Role	Student - 3
Module	3D Interactive Media
Submission date	12th of November 2026
GitHub	https://github.com/rashennimsara/TreasureIsland-GroupProject/tree/member3_characterControl

1. Introduction and project overview

This Island Treasure Hunter is a group-developed 3D interactive LEGO-style adventure game created using Unity 6. The project is based on a tropical island environment where a LEGO minifigure explores different areas, interacts with environmental objects, overcomes obstacles and progresses towards a hidden treasure. The island includes natural features such as trees, rocks, mountains, rivers and caves, creating an environment suitable for exploration and interactive gameplay.

My main contribution focused on developing the playable LEGO character and implementing its movement and animation systems. This involved creating a custom minifigure using Mecabricks, importing character assets into

Unity, implementing a physics based character controller and connecting character movement with animation states. I also worked on integrating the character into the group's main island environment to support navigation and interaction with gameplay objects.

2. My role and responsibilities

As Student 3, I developed the character controller system for Island Treasure Hunter. I designed the LEGO minifigure in Mecabricks, prepared its FBX animation files, and configured Rigidbody physics and CapsuleCollider components. Using C# in Unity, I wrote TestPlayerMovement.cs to handle camera-relative movement, smooth rotation, jumping and ground detection. I also built an Animator Controller to manage the standing, walking and sleeping animations, then integrated the player into the main island scene and tested its movement around the environment.

3. Development and implementation

3.1 LEGO character Modelling and Asset Import

I imported a LEGO character model into Unity and placed it beneath a TestPlayer object to keep the visible character separate from the main player object. I worked with Walk and Stand model or animation assets and adjusted the character's position and scale to better match the island environment and the breakable wall. I investigated the Capsule Collider and Rigidbody components because movement and collision behaviour depend on how the player object is configured. I also worked on keyboard movement and jumping and tested the character in the island setting. During these checks, the relative scale of the player and wall was important: if the character appeared too large, the cave puzzle would not look believable. Character setup was supporting work for the trigger test, not a claim that I completed the group's final animated character controller.

3.2 Physics-Based Character Controller

I created a CaveEntranceTrigger GameObject near the proposed cave entrance. I added a Box Collider and enabled Is Trigger so that the area could detect an entering object without acting as a solid wall. I worked with CaveTrigger.cs and used the Player tag to identify the test character. When the player entered the trigger area, a Console message indicated that the trigger had detected the character. This was a useful initial test because it confirmed that the trigger volume, player tag and script event were communicating. I adjusted the trigger's position to suit the planned cave entrance. The next step was to connect this detection to the breakable-wall response and verify the entire sequence in Play mode. A successful Console message alone does not prove that the complete wall explosion works.

3.3 Jumping Mechanics and Ground Detection

I imported a stone-wall model for testing and prepared a BreakableStoneWall object containing separate stone pieces, including Stone_01 through Stone_10. I worked with Box Colliders and Rigidbody components on the individual pieces so that they could participate in a physics-based scattering effect. I added or configured a WallExplosion script and connected the wall explosion component to the CaveEntranceTrigger setup. I also adjusted the wall's scale and placement in relation to the LEGO test character and the cave area. Separating the wall into pieces was necessary because one unbroken mesh would not behave like a brick array during an explosion. The scene and components were prepared for the intended interaction, but the provided work record does not confirm a reliable final scattering result. That result should be documented only after testing it in the actual project.

3.4 Animator Controller and Character Animation

I created an Animator Controller named PlayerAnimator to manage the LEGO minifigure's animation states.

The Animator Controller contains three main states: Stand, Walk and Sleep. Stand was configured as the default state, while transitions between animations were controlled using the boolean parameters isWalking and isSleeping. The Animator was connected to TestPlayerMovement.cs so that the character's animation could respond to movement and keyboard input. When the player begins moving, the character transitions from standing to walking. When movement stops, it returns to the standing animation. The sleeping animation can be activated using the Z key, and the character returns to an active state when movement or jumping is detected. This system connects the character's visual behaviour with player input, helping create a more responsive gameplay experience.

4. Individual Contribution to the Group Project

My contribution provided the main playable character system for Island Treasure Hunter. The custom LEGO minifigure, physics-based controller and animation state machine support the player's ability to explore the island environment. The character movement system also provides a foundation for other interactive features developed by the group. These include camera control, environmental triggers, obstacles and treasure-related gameplay. By separating character movement and animation functionality into reusable components, the player system can be further developed and improved during CW2. My work also involved coordinating with other group members to ensure that the character could be incorporated into the shared Unity project.

5. Testing, challenges and solutions

During development, several technical considerations were important for achieving stable character movement and animation behavior. One challenge was maintaining the character's balance when using Rigid body physics. To address this, rotation constraints were applied to prevent unwanted tipping, and Rigid body interpolation was enabled to improve movement smoothness. Another consideration was ensuring that the character moved relative to the camera rather than following fixed world directions. Camera-relative movement vectors and smooth rotation were used to address this requirement. Ground detection and jumping also required additional logic to handle different terrain conditions. Coyote Time, Jump Buffering and raycast based ground checks were introduced to improve responsiveness. Animation integration required the imported clips to share compatible Avatar settings and transition correctly between movement states. The character was tested in the island environment, although complete interaction testing with every gameplay component remains part of the group's ongoing integration work.

6. Individual Reflection and Future Improvements

Individual Reflection My main role was building the character controller and integrating it into the main scene. I learned how to handle player inputs, manage smooth character rotation, and configure colliders to prevent clipping. A key challenge was balancing the jump physics and gravity to make the platforming feel responsive and fun rather than floaty.

Future Improvements

- Advanced Mechanics: Adding a dash, crouch, or double jump.
- Smoother Animations: Using Unity's Animator Blend Trees for seamless transitions between walking and running.
- Better Camera: Integrating Cinemachine for a dynamic camera that avoids clipping through walls.
- Cleaner Code: Refactoring the movement script using a Finite State Machine (FSM) to easily add new states like swimming.

7. AI Assistance and Responsible Use

Number	Question Asked to Ai(Gemini , chatgpt)	AI Assistance Received & How it was Used
01	"How to smoothly transition between standing (idle) and walking animations?"	Ai explained how to use Animator Blend Trees and float parameters. I used this to blend animations seamlessly based on the character's movement speed.
02	"How to trigger a sleep animation and stop the player from moving?"	The AI suggested using an Animator Boolean and adding a flag in the script to lock inputs. I applied this to create a mechanic where the character sleeps and cannot walk until awakened.

03	"Why is my character stuck in the walk animation after I stop moving?"	Learned how to properly configure transition conditions and remove "Has Exit Time." I fixed the Animator so the character immediately returns to standing when input stops.

8. References and asset credits Appendix A: Evidence of individual contribution



Figure 1: Creating character test player in mechabriks.

credits Appendix B: Ai usage

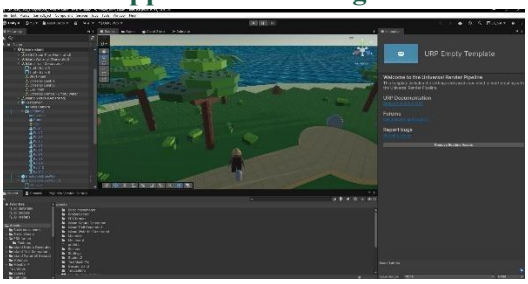
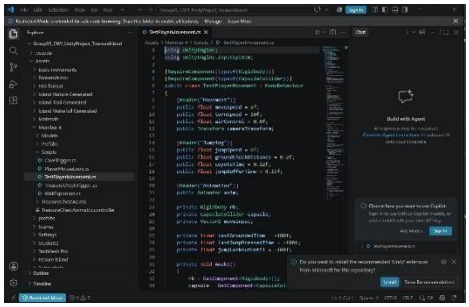
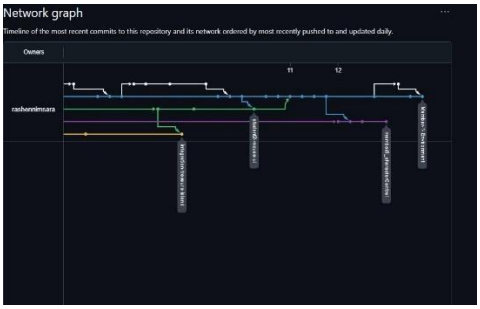


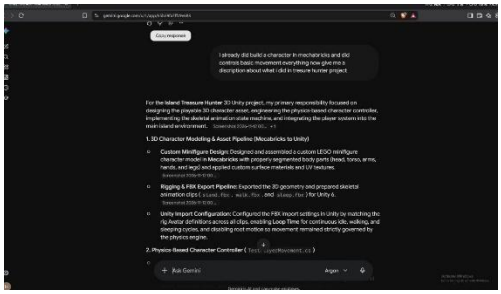
Figure 2: insert basic animations and project hierarchy.

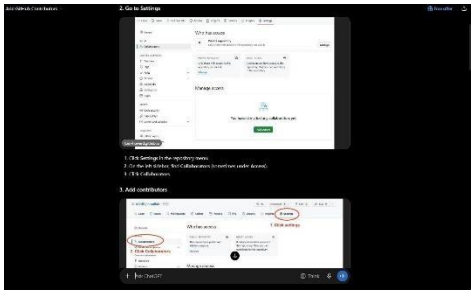


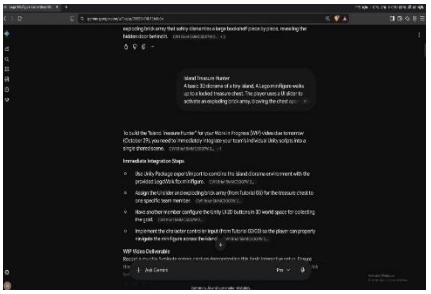
3 C# Script for basic movments



2Github network log







8. 1 References

Youtube references : https://youtu.be/mbuH_n5dl38 **(How To Animate 3D Character Movement In Unity)**
https://youtu.be/s5lRq6-BVcw **(How to work with humanoid animations in Unity)**

Unity Technologies (n.d.) *Animator.Play*. Available at: https://docs.unity.com/en-us/engine/6000.0/scriptreference/unityengine/animator/play (Accessed: 11 November 2026).

Unity Technologies (n.d.) *Creating text – TextMesh Pro*. Available at: https://docs.unity.cn/Packages/com.unity.ugui@2.0/manual/TextMeshPro/TMPObjects.html (Accessed: 11 November 2026). Unity Technologies (n.d.) *OnTriggerEnter(Collider)*. Available at: https://docs.unity.com/enus/engine/6000.5/scriptreference/unityengine/monobehaviour/ontriggerenter (Accessed: 11 November 2026).